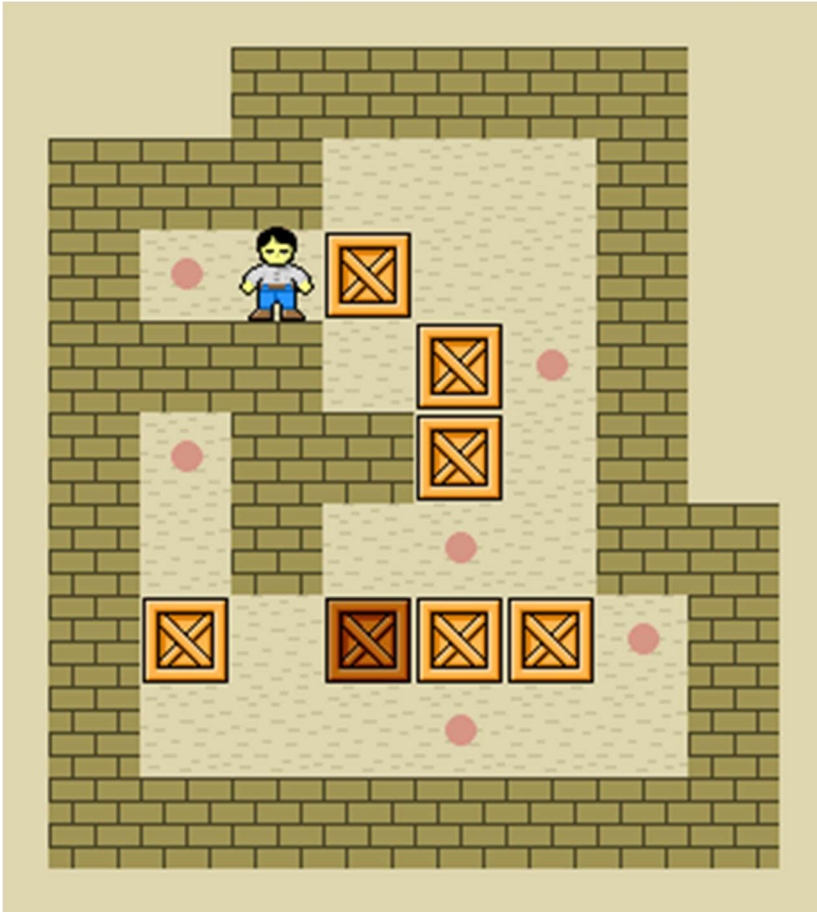




DOCUMENTACIÓN FINAL

Sistemas Inteligentes

DESCRIPCIÓN

En este documento podrá encontrar todo aquello relacionado con la resolución del problema Sokoban

SANTIAGO SÁNCHEZ-CELEMÍN
ARÉVALO

Grupo Laboratorio: C2

Nombre: Santiago Sánchez-Celemín Arévalo

Curso: 3º C

Asignatura: Sistemas Inteligentes

Grupo Laboratorio: C2

Contenido

1. Descripción del problema	2
2. Objetivo del Laboratorio	2
3. Detalles de resolución	3
a. Clase Búsqueda_algoritmo.py	3
b. Clase Descripción_tablero.py	3
c. Clase IDGenerador.py	3
d. Clase Manhattan.py	4
e. Clase Nodo.py	4
i. Método __init__	4
ii. Método Seleccionar_estrategia	4
iii. Método Print_estado	4
iv. Método Print_solucion	4
v. Método __lt__	4
f. Clase Sucesores.py	4
a. Método generar_sucesores	4
b. Método Cajas_en_objetivos	4
c. Método en_objetivo	4
g. Clase Tablero.py	5
a. Método __init__	5
b. Método descrifrar_cadena	5
c. Método control_errores_tablero	5
d. Método Posición_valida	5
h. Clase sokoban.py	5
a. Tarea 1	5
b. Tarea 2	5
c. Tarea 3	5
4. Manual de usuario	6
5. Opinión Personal	9
6. Video Manual de Usuario	9

Nombre: Santiago Sánchez-Celemín Arévalo

Curso: 3º C

Asignatura: Sistemas Inteligentes

Grupo Laboratorio: C2

1. Descripción del problema

Durante el Primer Semestre en la asignatura de Sistemas Inteligentes en la sección de prácticas, se ha llevado a cabo el desarrollo de un juego llamado sokoban, pero ¿Qué es sokoban?

Sokoban, es un juego clásico basado en un rompecabezas el cual fue inventado en Japón, el cual significa 'Encargado de Almacén', su objetivo es con un jugador empujar un número 'n' de cajas hasta un número 'z' de objetivos, concluyendo al situarse todas las cajas sobre un objetivo, ¿Parece fácil no?

Sokoban parece un juego aparentemente para niños, pero lo que no sabemos es que pueden surgir múltiples problemas mientras ejecutamos este, algunos de los problemas que puede ocasionar son:

- **Caja en un rincón:** Dado que una caja solo puede ser empujada y no arrastrada si un objetivo se queda en una esquina sería prácticamente imposible sacarla de ahí.
- **Una única caja:** Solo puedes empujar una sola caja lo que limita las estrategias a realizar.
- **Movimientos a realizar:** Todo depende de los límites del tablero, de la posición de las cajas y de la posición del jugador.
 - **Tablero:** Cuanto más grande el tablero mayor número de movimientos a realizar
 - **Cajas:** El número de cajas multiplica las posibilidades de movimientos

Podemos calcular su número de posibilidades máximas a realizar mediante un cálculo:

$$\binom{n}{b} * b! * n$$

**/ Siendo $\binom{n}{b}$ las combinaciones de b cajas en n espacios*

2. Objetivo del Laboratorio

El objetivo de este laboratorio está relacionado con **las clases teóricas** que hemos visto sobre búsquedas mediante árboles (como BFS, DFS, etc.). En particular, este juego de Sokoban lo hemos modelado utilizando estas **técnicas de búsqueda** aplicadas a un árbol, lo que nos permite explorar diferentes estrategias de resolución del problema.

Para entender mejor cómo se aplica esto, consideremos las mecánicas del juego. Cada movimiento posible en el juego se puede categorizar en **cuatro direcciones**: adelante, atrás, izquierda y derecha. Estas direcciones nos permiten generar un árbol de búsqueda donde cada nodo representa un estado del juego, y los movimientos posibles entre esos estados corresponden a las transiciones entre nodos en el árbol.

La clave de este modelo radica en la función objetivo del problema: Lograr que todas las cajas se coloquen en los objetivos. Es decir, la solución del problema consiste en encontrar un nodo dentro de este árbol en el que todas las cajas estén correctamente colocadas. Para hacer esto, debemos aplicar una estrategia de búsqueda que explore eficientemente el árbol de estados, buscando un camino desde el estado inicial hasta el estado final donde se cumpla la condición de que todas las cajas están en sus objetivos.

Nombre: Santiago Sánchez-Celemín Arévalo

Curso: 3º C

Asignatura: Sistemas Inteligentes

Grupo Laboratorio: C2

Por lo tanto, la estructura de árbol, combinada con las estrategias de búsqueda como BFS o DFS, nos permitirá explorar las diferentes posibilidades de movimientos y transiciones, hasta encontrar el nodo solución que represente el estado final del juego. De esta manera, este ejercicio no solo pone en práctica las técnicas de búsqueda estudiadas en teoría, sino que también nos ofrece un enfoque práctico para resolver un problema complejo de manera estructurada y eficiente.

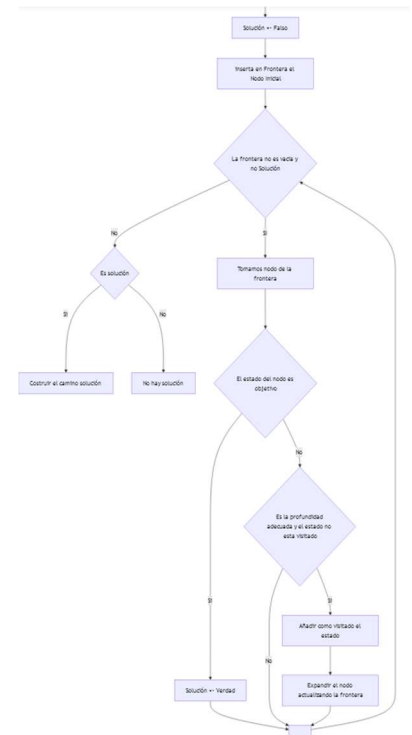
3. Detalles de resolución

Para poder exponer mejor los detalles de solución lo realizaremos de tal forma que sigamos cada una de las clases desarrolladas a lo largo de las prácticas

a. Clase Búsqueda_algoritmo.py

Esta clase contiene el algoritmo el cual desarrolla el árbol, para el desarrollo de este seguimos la imagen subida a campus virtual. Para mejorar la exposición del desarrollo de este enumeramos cada uno de los pasos:

- i. Definimos los elementos necesarios para el desarrollo del algoritmo
- ii. Generamos el nodo inicial (Nodo padre)
- iii. Lo almacenamos en la lista frontera
- iv. Iniciamos un bucle si existen nodos en la lista frontera o si no se ha encontrado solución
- v. Sacamos al nodo de la frontera
- vi. Comprobamos si todas las cajas se encuentran en algún objetivo.
 1. Si están todas en objetivos entonces asignamos el nodo final y ponemos la variable la variable booleana a True para que no itere más
- vii. Comprobamos si la profundidad dada es menor a la establecida y si el nodo no está en la lista visitados
 1. Si no está visitado continuamos
 2. Si esta visitado volvemos atrás
- viii. Generamos los sucesores del nodo sacado
- ix. Generamos un nodo Hijo
- x. Almacenamos en la lista frontera el nodo hijo



b. Clase Descripción_tablero.py

Esta Clase es la encargada de imprimir todos los parámetros definidos, es decir recoge todos los parámetros y los imprime en el orden correspondiente

c. Clase IDGenerador.py

Será el encargado de recoger la tupla con la posición del jugador y de las cajas para posteriormente calcular mediante la librería 'haslib' con md5 la codificación y devolución de un Id.

Nombre: Santiago Sánchez-Celemín Arévalo

Curso: 3º C

Asignatura: Sistemas Inteligentes

Grupo Laboratorio: C2

d. Clase Manhattan.py

La clase Manhattan solo será utilizado por las dos estrategias (GREEDY y A*) dado que son aquellas que utilizan heurística, esta calcula la posición más cercana de una caja hasta un objetivo, para ello tomamos mano de la fórmula proporcionada en Campus. Este recorre cada caja para cada uno de los objetivos calculando la distancia Manhattan, una vez va generando se va quedando con la distancia menor

$$Manhattan(fila, columna) = |Tfila - Bfila| + |Tcolumna - Bcolumna|$$

e. Clase

Nodo.py

Esta clase definirá cada uno de los nodos que desarrollarán el camino, cada nodo contará con una serie de atributos, nombrados en la Tarea 3

i. Método __init__

Este método inicializa cada uno de los distintos nodos con todos sus atributos pertinentes

ii. Método Seleccionar_estrategia

Será el método encargado de calcular el valor de cada uno de los nodos mediante una de las estrategias seleccionada en la línea de comandos

iii. Método Print_estado

Será el encargado de devolver el estado para la impresión de los id's en MD5

iv. Método Print_solucion

Se encargará de generar la solución acorde a lo pedido en el enunciado, para ello ha sido necesario importar la librería 'Decimal', ya que la función 'round' en Python redondea sus números, a la baja, es decir, si tenemos 0,565, su resultado sería 0,56. Pero nosotros queremos que el resultado sea 0,57 para ellos he importado dicha librería

v. Método __lt__

Encargado de comprobar en primer lugar por valor y en caso de empate lo comprobará por su Id del nodo

f. Clase Sucesores.py

En esta clase podremos encontrar como nuestro jugador es capaz de desplazarse por las distintas posiciones en 4 diferentes movimientos, en el caso de que encuentre en un movimiento válido, nuestro jugador se moverá, almacenando la nueva posición, pero en el caso de que su movimiento coincida con la posición de una caja, se moverá la caja, y la posición se almacena, teniendo en cuenta que la letra del movimiento será en mayúscula. Devolverá: Tripleta (Movimiento, estado, costo)

a. Método generar_sucesores

Este método se encargará de generar los nuevos posibles movimientos a realizar, tanto del jugador solo, como del jugador con las cajas

b. Método Cajas_en_objetivos

Se ocupará de comparar si cada una de las cajas se encuentra en cada uno de los diferentes objetivos, iterando una caja sobre todos los objetivos y así con todas las cajas y todos los objetivos

c. Método en_objetivo

Comprobamos si una sola caja, se encuentra sobre un solo objetivo

Nombre: Santiago Sánchez-Celemín Arévalo

Curso: 3º C

Asignatura: Sistemas Inteligentes

Grupo Laboratorio: C2

g. Clase Tablero.py

Esta clase será la encargada de obtener y comprobar todos los datos procedentes de la línea de comandos, mediante la cual será necesario decodificar para obtener el tablero de nuestro 'Sokoban', para ello utiliza unos métodos como son:

a. Método __init__

Cuenta con todos los atributos con los que se inicializará nuestro tablero

b. Método descifrar_cadena

Este método será el encargado de descifrar y desenvolver la cadena recibida, identificando cada uno de los elementos necesarios para el desarrollo del juego

c. Método control_errores_tablero

Será el encargado de comprobar que en nuestro tablero se encuentran algunos de los elementos principales, como son el jugador, las cajas o los objetivos

d. Método Posición_valida

Se encargará de comprobar si la posición en la que nos encontramos es una caja o es una pared/Límite

h. Clase sokoban.py

En ella encontraremos lo necesario para poder inicializar nuestro código, es decir, es el 'main' de nuestro programa, este se divide en varias condiciones de if-elif, las cuales son:

a. Tarea 1

Esta condición se encargará de que si recibe un T1 en la línea de ordenes ejecute lo conveniente a la Tarea T1, todo ello con un control de errores, refiriéndose al tamaño de la cadena

b. Tarea 2

Está condición puede tener dentro de ella dos distinciones:

- **T2S:** En la que imprime una lista de sucesores que puede tener un jugador con o sin caja, para ello eliminamos los espacios para realizar la codificación MD5 de forma correcta, posteriormente formalizo el mensaje en la estructura solicitada
- **T2T:** En el que comprobaremos que todas nuestras cajas se encuentran en alguno de los distintos objetivos existentes, imprimiendo dos valores: TRUE/FALSE

c. Tarea 3

En esta condición inicializaremos el método 'algoritmo' de la clase 'Busqueda_algoritmo', en la que nos ofrecerá el último nodo, es decir, el nodo solución. Por ello debemos de iterar desde ese nodo, buscando conocer los padres de cada uno de los hijos y así con todos hasta llegar al último que no tendrá padre y será 'None', posteriormente se le da la vuelta a la cadena para dejarla de forma correcta ordenada

Nombre: Santiago Sánchez-Celemín Arévalo

Curso: 3º C

Asignatura: Sistemas Inteligentes

Grupo Laboratorio: C2

4. Manual de usuario

- **Requisitos:**

Tener instalado **Visual Studio Code**.

Tener el repositorio del programa **Sokoban** clonado en tu ordenador.

*Si no tiene el repositorio Clonado use en la terminal: git clone

[https://github.com/SI2024-ESI/sokoban-](https://github.com/SI2024-ESI/sokoban-SantiagoSanchezCeleminArevalo.git)

[SantiagoSanchezCeleminArevalo.git](https://github.com/SI2024-ESI/sokoban-SantiagoSanchezCeleminArevalo.git)

- **Pasos para ejecutar el programa:**

- **Abrir Visual Studio Code:**

Haz doble clic en el ícono de Visual Studio Code o ábrelo desde el menú de inicio.

- **Abrir la carpeta del proyecto:**

- Dentro de Visual Studio Code, selecciona la opción File (Archivo) en la barra superior.
- Selecciona Open Folder (Abrir carpeta)
- Busca la ubicación donde está clonada la carpeta del proyecto Sokoban.
- Haz clic en Seleccionar carpeta para abrirla en el editor de 'Code'.

- **Abrir el terminal:**

Presiona Ctrl + ~ (Windows/Linux) para abrir el terminal en Visual Studio Code.

- **Ve a la carpeta src:**

En el terminal, ejecuta el siguiente comando:

- `cd /sokoban/src/`

- **Ejecutar el programa:**

Para ejecutar el programa puede tener varias Opciones de ejecución:

- **T1:**

En esta opción podrá ver la descripción del mapa establecido. Puede utilizar esta línea de instrucciones:

- `Python3 ./sokoban.py T1 -l '#####\n#### .# #\n#### # $.\n##### $# #\n#@$. ##\n# ### #\n# ## #\n#####'`
- Solución: [1]

- **T2:**

En esta opción podrá generar los sucesores iniciales del árbol (T2S) o comprobar si todas las cajas se encuentran en algún objetivo (T2T)

- **T2S:**

- `Python3 ./sokoban.py T2S -l '#####\n#.$@\n#$. #\n#####'`

- Solución:[2]

- **T2T:**

- `Python3 ./sokoban.py T2T -l '#####\n#*@\n# * #\n#####'`

- Solución: [3]

Grupo Laboratorio: C2

```
PS C:\Users\USUARIO\Desktop\sokoban-SantiagoSanchezCeleminArevalo\sokoban\src> Python3 ./sokoban.py T3 -1 '#####\n#### @# \n#####  
# \n#### $# #\n# $ . .# \n# ## $# \n# ## $# \n# ## $# \n#####' -s A* -d 100  
#####\n#####\n#####\n0,4256074849CF9525EB4D8767D6A4554,None,NOTHING,0,0.00,3.00,3.00  
1,778087FA9A8D32F897815975B7DC7AD1,0,d,1,1.00,3.00,4.00  
4,8E60FC1F133831FC957E4727DE23C9F,1,r,2,2.00,3.00,5.00  
10,2A6AC36AA89982E5446EEAB80B602B32,4,r,3,3.00,3.00,6.00  
21,1FA78F175256804B0F448B8179A78E9D,10,r,4,4.00,3.00,7.00  
38,C08C014C85F491B8FF6ED1E4A1667D87,21,d,5,5.00,3.00,8.00  
59,58887FC56CF31D87457D041E878799F1,38,d,6,6.00,3.00,9.00  
79,03365DC3BAB9C72CF3CD38F8A053803,59,D,7,7.00,2.00,9.00  
94,359BA3ABA57FAD8AA33A11F515E9AE0,79,u,8,8.00,2.00,10.00  
110,88CE2DF63D44ED445E8FA10F77D199,94,u,9,9.00,2.00,11.00  
137,7784E2F1FFF41081FE380E7FE1B3B0E,110,u,10,10.00,2.00,12.00  
182,9580DA4D8835D0821AFA2BE2BBD7408,137,1,11,11.00,2.00,13.00  
231,8140222CD1FA2703C71DCC944DDC707B,182,1,12,12.00,2.00,14.00  
277,F44A6A2305C7C8D48021E85116315203,231,1,13,13.00,2.00,15.00  
331,87605D0C330846C39203A8E6C55B0CDA,277,D,14,14.00,1.00,15.00  
349,0756822CD900FEA7730F106C0E4A02D2,331,1,15,15.00,1.00,16.00  
404,5547ABDF711DF5B5778428632159C2A3,349,d,16,16.00,1.00,17.00  
470,54E100912734671ACB05E19A09E1D8C2,404,1,17,17.00,1.00,18.00  
544,85D0DC47F4D6211647CA886709522113,470,1,18,18.00,1.00,19.00  
619,063127A0AD8E9B199FF19604466DE5EF,544,d,19,19.00,1.00,20.00  
683,A426A3A870169BD07D6F7897184129EA,619,1,20,20.00,1.00,21.00  
749,D2ECDC44887ABFE45CBF80B8EDD8755,683,1,21,21.00,1.00,22.00  
837,38274571BB37CB7DB21FE1ECA71BCDF5,749,u,22,22.00,1.00,23.00  
927,97F868AE7C23DAD9E4580FF88F304DEB,837,R,23,23.00,0.00,23.00
```

```
PS C:\Users\USUARIO\Desktop\sokoban-SantiagoSanchezCeleminArevalo\sokoban\src> Python3 ./sokoban.py T3 -l '#####n#### @# #\n#### #
#n#### $# #\n# $. .## #\n# ### $#\n# ### #\n#####' -s GREEDY -d 100
#####\n####@##\n#####\n#####$\n#$. .##\n#####\n#####. #\n#####
0,4256D74849CF5925E8AD8767D6A4554,None,NOTHING,0,0.00,3.00,3.00
1,778B7FA9A8D32FB97815975870C7AD1,0,d,1,1.00,3.00,3.00
5,5DEB150CB8C5A78C37B58CD4AE548131,1,D,2,2.00,2.00,2.00
6,93A73EE35A8403CA9D96927C6923A4,5,u,3,3.00,2.00,2.00
9,1871DAEA749BDAE2D59D580801B78B79,6,r,4,4.00,2.00,2.00
16,0354C50FC28B19422081BC161495ABE8,9,r,5,5.00,2.00,2.00
26,CF32651C1D49A77805D085D413249ADAC,16,r,6,6.00,2.00,2.00
39,0688B8228BCF09D7DF122D2FE7C94519,26,d,7,7.00,2.00,2.00
49,14AE480E895D96A2BE06CFE9025DC69,39,d,8,8.00,2.00,2.00
55,452B9F4F9980FAC94E6523B6E617287E,49,D,9,9.00,1.00,1.00
56,FB36E1534CFF5D41365CAD6992638FBA,55,u,10,10.00,1.00,1.00
58,77258797175C0EED5A4CD4D808C9DC4E,56,u,11,11.00,1.00,1.00
63,DE538B80E224DC93760DE8C9FEB8AE89,58,u,12,12.00,1.00,1.00
72,85F7DE134D5BEDFD2D0837A1DC2289773,63,l,13,13.00,1.00,1.00
82,0CD149DF5AD3692712290479A2DC4539,72,l,14,14.00,1.00,1.00
86,DA007F206D70FA06AC9A023BD01CA64F,82,l,15,15.00,1.00,1.00
89,87605D0C330846C39203A8E6C5580CDA,86,d,16,16.00,1.00,1.00
93,0756822CD900FEA7730F106C0E4A0D2D,89,l,17,17.00,1.00,1.00
97,5547ABDF711DF585778428632159C2A3,93,d,18,18.00,1.00,1.00
102,54F100912734671ACB05E19A09E1D8C2,97,l,19,19.00,1.00,1.00
110,85D0DC47F4D6211647CA886709522113,102,l,20,20.00,1.00,1.00
112,063127A0AD8E9B199FF19604466DE5FF,110,d,21,21.00,1.00,1.00
116,A426A3A870169BD07D6F7897184129EA,112,l,22,22.00,1.00,1.00
121,D2ECDC44887ABEFE45CBF80BBEDD8755,116,l,23,23.00,1.00,1.00
125,38274571B837CB7D821FE1ECA71BCDF5,121,u,24,24.00,1.00,1.00
130,97F868AE7C23AD9DE4580FF8F8F3D4EBE,125,R,25,25.00,0.00,0.00
```

Esto es respecto a mi ordenador, puede depender según las características de su ordenador

```
real    1m1,313s
user    1m0,839s
sys     0m0,488s
```

Nombre: Santiago Sánchez-Celemín Arévalo

Curso: 3º C

Asignatura: Sistemas Inteligentes

Grupo Laboratorio: C2

5. Opinión Personal

En mi opinión durante el desarrollo de esta práctica he adquirido conocimientos prácticos de lo visto durante las clases teóricas, especialmente en todo aquello relacionado con los algoritmos de búsqueda en arboles en algunas de sus estrategias, así como la implementación de sus heurísticas, lo que me ayudó a profundizar aun más en la programación y la lógica.

Durante el desarrollo de esta práctica me enfrenté a un reto respecto a complejidad, dado que si aumentamos el número de cajas y el tamaño del tablero su complejidad aumenta, incluso pudiendo llegar a poner nuestro ordenador en condiciones extremas debido a su alta complejidad. A su vez aprendí como la heurística aplicada en ciertas estrategias me ayudó a comprender como puede llegar a mejorar la eficiencia de la resolución de ciertos problemas en comparación de otras estrategias.

A pesar de esto hubo ocasiones en las que el código presentaba ciertas complicaciones, pongo el error que tuve al empezar a programar inicialmente utilizando una única clase, dado que encontrar los distintos métodos o hacer pasar parámetros entre ellos me hacía utilizar demasiadas variables globales, para solucionar esto y poder localizar mejor y comprender mejor lo realizado me decidí hacerlo mediante clases, lo que solucionó bastante estos errores.

En resumen, esta práctica no solo me ayudó en la comprensión de los algoritmos de búsqueda y sus estrategias, sino que también me ayudó a comprender la importancia de la planificación y la importancia de modularizar y estructurar el código. Para terminar, puedo decir que los resultados obtenidos respecto a lo aprendido y a el consumo de tiempo llevado en el desarrollo de esta práctica ha sido de gran importancia en el mundo actual, por poner un ejemplo, estos conocimientos son de gran utilidad en el desarrollo de videojuegos.

6. Video Manual de Usuario

Para mejor explicación en el manual desarrollo un video explicativo:

- <https://youtu.be/AQUCr93CiSc>